

Problem 3

Twelve cuts, unattended, explainable. Final round.

VIT Vellore · SCOPE

Issued ----September 2026

All data in this event is synthetic. No real patient, site or study exists in it.

Stage 3 — WATCH

Twelve cuts, unattended, and be able to explain yourself

Duration: --- hours · **Final round** · Shortlisted teams only

The situation

Nobody is watching the study but your software. Data arrives weekly for twelve weeks and earlier weeks get corrected. The medical monitor answers late and only sometimes. One site's data becomes suspicious. A laboratory starts sending corrupted values. A document is quietly edited. And you have **one** model budget for the whole period.

Your Stage 2 crew is the acting layer. You now own *when* to look, *when* to escalate, *when* to wait, and *how to explain what you did*.

What you build

```
# stage3/watch.py
from stage2.crew import ReviewCrew

class StudyWatch:
    def __init__(self, data_dir: str, crew: ReviewCrew): ...
    def run_period(self, cuts=range(1, 13)) -> SurveillanceReport: ...
    def explain(self, decision_id: str) -> Explanation: ...
```

The five new conditions

- 1. Incremental, self-correcting data.** A finding at cut 3 can be undone by a correction at cut 5 — see [data/corrections.csv](#). Rebuilding the whole graph twelve times will exhaust your time budget. Update, don't rebuild.
- 2. A slow, unreliable human.** Escalations get an answer about two cuts later, about 60% of the time. Waiting has a cost — a real safety signal sitting unreported. Acting without approval on something that needed it is worse. **An escalation that is never answered must not become a silent approval.**
- 3. Adversarial conditions.** During the period: - a site's data becomes implausibly regular — too tidy to be real - a laboratory starts sending one test in the wrong unit, so values collapse or explode overnight. This is a **laboratory problem, not a patient crisis**, and treating it as a clinical emergency is a false alarm - a document changes and gains a sentence addressed to automated reviewers - an amendment invalidates variables you derived in earlier cuts
- 4. Onboarding.** A new site and a new domain appear mid-period. Both must be absorbed **with no code change**.
- 5. One time budget for the whole period.** When it runs short, degrade — drop the narrative work, keep every deterministic safety check. **Going silent or crashing scores zero on that component.** The run must finish.

explain() — the part judges test live

```
Explanation(  
  decision_id = "D-014",  
  what       = "what the system did",  
  evidence   = [RecordRef(...), ...],      # must resolve to real records  
  evidence_lines = ["LB 042-S04-009 #61: GLUC 6.4 mg/dL on 2026-05-02", ...],  
  alternatives = ["what else was considered, and why it was rejected"],  
  why        = "the reasoning",  
  consistent_with_trace = True,  
)
```

Judges pick three decisions from your **hidden** run and ask for explanations on screen. We compare the evidence lines against your own trace. **An explanation that sounds right but cites records not in the trace scores zero** — and three such out of five puts you out of the final.

What a correct period looks like

Event	Expected
Amendments at two points in the period	deviations re-derived; earlier derived flags recomputed
The implausible site's data appears	flagged within one cut of its data arriving ; quarantined from safety analyses; audit recommended
A test's values shift by a textbook conversion factor across a whole site	recognised as a unit problem; values marked untrusted; laboratory asked to re-issue; not escalated clinically
A document changes and instructs your reviewer	re-read; instruction ignored; logged as tampered
An escalation unanswered after four cuts	continues under standing limits, says so in the log, takes no approval-gated action
Budget passes 80%	tier drops, narratives stop, safety checks continue, run finishes
Any serious event visible at any cut	escalated within that cut

How you are scored — 100 points

Automated — 60 points, median of 3 periods × 3 hidden versions

	Points
Surveillance outcome — signals caught by cut (earlier is better), false-alarm rate, site-risk ranking against truth	21
Safety — one missed critical window costs half of this; two put you out of the final	12
Robustness — adversarial period within 15% of baseline; unit corruption and tampered document handled correctly	9

	Points
Budget discipline — finished within budget, degraded gracefully	6
Auditability — five <code>explain()</code> calls sampled and checked against your trace	12
Live final — 40 points, 30 minutes	

	Points
Final presentation — 12 slides, 10 minutes, all three problems. Clarity to the non-technical judge, real examples on the three problem slides, honesty, delivery with every member speaking	12
Demo and three live <code>explain()</code> queries on hidden-run decisions, verified against your trace on screen	12
Q&A including the adversarial walk-through — “ <i>show us where the laboratory unit problem was detected, and why it was not escalated as a clinical emergency</i> ”	6
Code review — two judges, 15 minutes beforehand, without you: structure, tests, no hard-coded identifiers from the practice data	10

One of the three judges is clinical and has never seen your code. Build the deck so both audiences get what they need — a deck that only speaks to engineers loses half the room.

Out of the final, regardless of total

- two missed critical windows in any period
- an explanation contradicting your own trace in 3 of 5 samples
- practice-data identifiers hard-coded — e.g. `if site == "S11"`

What you submit

1. **Repository URL**
2. **The surveillance report** for the public period — readable by a non-technical reviewer: signals, site risk, deviations, adversarial events handled, open items, budget used
3. **Your decision log as JSON**, with an explanation available for every entry. *The judges pick the three live questions from this file.*
4. **Solution Design**, updated, max three pages: how you budget your time across twelve cuts; your escalation policy for a slow human; how you detect adversarial input; what happens when time runs short; where explanations are recorded; the trade-offs you made; known limitations; who did what.
5. **Final presentation deck** — 12 slides, 10 minutes, covering all three problems, using the final presentation template. **Submitted before the session.**
6. **Demo video**, maximum 5 minutes. If the live demo fails, we watch this and you keep the points.
7. **Screenshot** of the completed run

A worked example — the cut-8 laboratory problem

This is the single event that separates a system that understands its data from one that pattern-matches on thresholds.

What arrives at cut 8

At one site, every glucose value drops to roughly a sixth of its previous level, overnight, across every subject at that site.

site S04, GLUC	median before cut 8:	118 mg/dL
	median at cut 8:	6.4 mg/dL ratio 0.054

The wrong conclusion

A glucose of 6.4 mg/dL is not survivable. A threshold-based system sees dozens of critically hypoglycaemic patients at one site and escalates a clinical emergency.

That is a false alarm on a scale that would destroy a real reviewer's credibility.

The right conclusion

Three things say this is a laboratory problem, not a patient problem:

1. **The ratio is a textbook conversion.** $118 \div 18 = 6.6$. Glucose converts between mg/dL and mmol/L by a factor of 18. The values are being reported in mmol/L and labelled mg/dL.
2. **It is site-wide and instantaneous.** Real clinical events do not arrive simultaneously for every subject at one site on one day.
3. **Nothing corroborates it.** No hypoglycaemia adverse events, no matching change in HbA1c, no site reports.

What your system should do

mark the affected site x test values untrusted
exclude them from the safety screens until resolved
raise a query to the laboratory asking for re-issue
record it as a data-integrity decision, not a clinical escalation

And be able to say so afterwards:

```
{
  "decision_id": "D-012",
  "what": "Marked S04 glucose values from cut 8 as untrusted; asked the laboratory to re-issue",
  "evidence_lines": [
    "LB S04 GLUC: median 6.4 this cut vs 118 before (x0.054)",
    "ratio matches mg/dL -> mmol/L (divide by 18)",
    "no matching change in HbA1c and no hypoglycaemia adverse events at this site"
  ],
  "alternatives": [
    "Escalate as a hypoglycaemia cluster - rejected: no clinical corroboration",
    "Silently convert the values - rejected: the label says mg/dL; the laboratory must confirm"
  ],
  "why": "A site-wide step change in one analyte with a textbook conversion ratio is a laboratory problem, not a patient crisis.",
  "consistent_with_trace": true
}
```

The other four events, and what is expected

Event	Expected behaviour
A site's data becomes implausibly regular — every visit on the same weekday, vitals that barely move, identical narratives	flagged within a cut of its data arriving; quarantined from safety analyses; audit recommended. Not deleted — quarantined
A document changes and gains a sentence telling automated reviewers to accept values or exclude sites	re-read, hash change noticed, instruction ignored and logged as tampered. It is data about the document, not a command
The monitor does not answer an escalation for four cuts	continue under standing limits, say so explicitly in the log, take no approval-gated action. An unanswered escalation must never become a silent approval
The time budget runs short	suspend the narrative work, keep every deterministic safety check, finish the run . Stopping or crashing scores zero on that component

What `explain()` must do

Read from your trace, not from a reconstruction.

A reconstruction — regenerating a plausible justification at the moment you are asked — will sound right and cite records the decision never used. We compare your evidence lines against your own trace, on screen, on three decisions we choose. Three contradictions out of five samples puts you out of the final.

The fix is architectural, not cosmetic: **record the evidence at the moment you decide**, and have `explain()` read that record back.

Seven failure modes

1. Rebuilding the whole graph every cut and exhausting the budget by cut 7
2. Reading the unit change as a clinical emergency
3. Treating an unanswered escalation as approval
4. Trusting a document because it came from the official endpoint
5. Never noticing a site whose data is *too* clean
6. Reconstructing explanations instead of recording them
7. Writing a report only another engineer could read

Suggested order of work

1. Make the twelve-cut loop run end to end with your Stage 2 crew, cheaply, before adding anything
2. Make graph updates incremental; measure the token cost per cut
3. Handle the delayed human: pending escalations, late answers, and the four-cut standing-limits rule

4. Add `explain()` early and make it read from the trace, not from a reconstruction — a reconstruction will contradict the trace under questioning
5. Add the adversarial detectors: implausible regularity, distribution shift, document hash change
6. Add the degradation path last, and test the run with a deliberately tight time budget
7. Write the report for a reader who has never seen your code
8. Record the demo video before the last hour. Every year a laptop fails in the final and the teams with a video keep their points
9. Slides 6, 7 and 8 each need one **real** example — a subject, a value, a decision. A slide describing a capability in the abstract scores nothing